



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Dipartimento di Informatica – Scienza e Ingegneria
Corso di Laurea Triennale in Ingegneria Informatica

Estensione e redesign di un simulatore web per l'architettura DLX

Relatore:
Prof. Stefano Mattoccia

Presentata da:
Marco Tartaglione

Sessione Ottobre 2026
Anno Accademico 2025/2026

A mio padre per la curiosità
A mia madre per l'ostinazione
A mio fratello per la bontà
Ai miei nonni per la saggezza

Indice

Introduzione	1
1 Le architetture DLX e RISC-V	2
1.1 DLX	2
1.1.1 Registri	2
1.1.2 Memoria	3
1.1.3 Istruzioni	4
1.1.4 Pipelining	5
1.2 RISC-V	6
1.2.1 Origine e Mantenimento	6
1.2.2 Caratteristiche Architettureali Principali	6
1.2.3 Utilizzo Industriale	7
2 Ammodernamento Tecnologico	8
2.1 TypeScript	8
2.1.1 Caratteristiche Principali e Architettura	9
2.1.2 Da Typescript 3.5.3 a Typescript 5.8.3	9
2.2 Angular	10
2.2.1 Caratteristiche Principali e Architettura	10
2.2.2 Da Angular 8 ad Angular 20	11
2.3 CodeMirror	12
2.3.1 Caratteristiche Principali e Architettura	12
2.3.2 Da ngx-codemirror a CodeMirror Nativo	13
3 Redesign e sviluppo nuove funzionalità	14
3.1 Redesign	14
3.1.1 Memoria	14
3.1.2 Componenti <i>Device</i>	17
3.1.3 Interprete e Registri	20

3.2	Sviluppo delle Nuove Funzionalità	23
3.2.1	Comandi di Esecuzione e Scorciatoie da Tastiera	23
3.2.2	Starting Script	23
3.2.3	Sistema di Segnalazione Anomalie	23
3.3	Valutazione delle Prestazioni e Ottimizzazione della Memoria	24
4	Interfaccia Grafica	26
4.1	Palette e Identità Visiva	27
4.2	Pulsanti e Controlli	27
4.3	Registri	30
4.4	Dialog	30
5	Sviluppi Futuri	32
	Conclusioni	34
	Bibliografia	35

Introduzione

Il presente lavoro di tesi si propone di ottimizzare e, per alcune componenti chiave, riprogettare il simulatore web per le architetture *DLX* e *RISC-V*. Il progetto si inserisce in un percorso di sviluppo incrementale, avviato e progressivamente espanso dai precedenti lavori di laurea dedicati alla piattaforma [1–5].

Il simulatore nasce con l’obiettivo di non limitare la comprensione del funzionamento delle architetture alla sola astrazione teorica, bensì di mettere a disposizione uno strumento dettagliato, preciso e interattivo. Sviluppato per il corso di Architetture dei Calcolatori T dell’Università di Bologna, il software si rivolge principalmente agli studenti del corso stesso; per questo motivo, lo scopo fondamentale di questa tesi consiste nel correggere le anomalie di funzionamento, eliminare le funzionalità fuorvianti o errate e semplificare l’esperienza d’uso complessiva.

L’elaborato si articola come segue: dopo un’iniziale analisi teorica delle architetture simulate (con una particolare attenzione sul modello *DLX*), la trattazione approfondisce l’ammodernamento delle tecnologie adottate, la correzione e riprogettazione delle componenti critiche e, infine, il rinnovamento dell’interfaccia e dell’esperienza utente.

Capitolo 1

Le architetture DLX e RISC-V

1.1 DLX

L'architettura DLX (*Deluxe*) è stata introdotta nel 1990 da John L. Hennessy e David A. Patterson [6]. È un'architettura *RISC* a 32 bit basata sul modello di Von Neumann e su un principio di accesso alla memoria di tipo *load/store*. È stata concepita come una sintesi e un'evoluzione ideale dei progetti Stanford MIPS e Berkeley RISC, con l'obiettivo di semplificare l'implementazione del *pipelining* a 5 stadi (Fetch, Decode, Execute, Memory, Write Back).

Progettata prevalentemente per scopi didattici, la struttura del *DLX* offre una *Instruction Set Architecture* (ISA) regolare a lunghezza fissa (32 bit), garantendo un modello lineare e trasparente che ne fa uno degli standard di riferimento più utilizzati nei corsi universitari di architettura dei calcolatori.

1.1.1 Registri

L'architettura *DLX* gestisce sia registri visibili all'assembly (*programmer-visible*), sia registri di controllo e di interfaccia con il sistema. La memoria interna include 32 *General Purpose Register* (GPR) a 32 bit (denominati da R0 a R31). Tra questi, due presentano un comportamento o un impiego particolare:

- **R0**: il cui valore è cablato permanentemente a zero (*hardwired zero*). Qualsiasi operazione di scrittura verso R0 viene ignorata, facilitando la sintesi di istruzioni di spostamento o azzeramento;
- **R31**: utilizzato dall'hardware per memorizzare l'indirizzo di ritorno durante l'esecuzione delle istruzioni di salto a sottoprogramma (*Jump and Link*, JAL e JALR).

Oltre ai GPR, l'architettura si avvale di registri speciali di controllo, di stato e di interfaccia con il bus di memoria:

- **Program Counter** (PC): registro a 32 bit che contiene l'indirizzo dell'istruzione corrente o della successiva istruzione da eseguire;
- **Interrupt Address Register** (IAR): memorizza l'indirizzo del programma a cui riprendere l'esecuzione a seguito della gestione di un'interruzione o di una trappola (*trap*);
- **Memory Address Register** (MAR): registro di servizio che mantiene l'indirizzo di memoria verso cui effettuare l'operazione di lettura o scrittura;
- **Memory Data Register** (MDR): registro di transito che contiene il dato a 32 bit (*word*) appena letto dalla memoria o in attesa di esservi scritto.

1.1.2 Memoria

L'architettura *DLX* gestisce uno spazio di indirizzamento a 32 bit, permettendo di accedere a una memoria fisica fino a 4 Gigabyte (2^{32} byte). Ciascun byte all'interno della memoria è singolarmente indirizzabile (Figura 1.1); tuttavia, il sistema supporta anche operazioni di lettura e scrittura su *half-word* (16 bit) e *word* (32 bit). Per semplificare la struttura dell'hardware e ottimizzare i trasferimenti, l'accesso alla memoria rispetta un vincolo di allineamento stringente: non è pertanto consentito accedere ad indirizzi non allineati alla dimensione del dato richiesto.

A livello hardware, i due bit meno significativi dell'indirizzo (BA1–BA0) non vengono emessi individualmente sul bus indirizzi; il processore li decodifica internamente per generare quattro segnali di controllo dedicati, denominati *Byte Enable* (BE0–BE3), ciascuno dei quali abilita selettivamente uno specifico byte all'interno della parola indirizzata durante l'operazione di lettura o scrittura. Di conseguenza, il bus indirizzi effettivamente emesso dal processore (BA31–BA2) è composto da soli 30 bit (si veda lo schema architetturale in Figura 1.1).

Infine, nella sua configurazione didattica fondamentale, la struttura del *DLX* non prevede livelli intermedi di memorizzazione tra i registri del processore e la memoria principale, risultando nativamente priva di memoria cache.

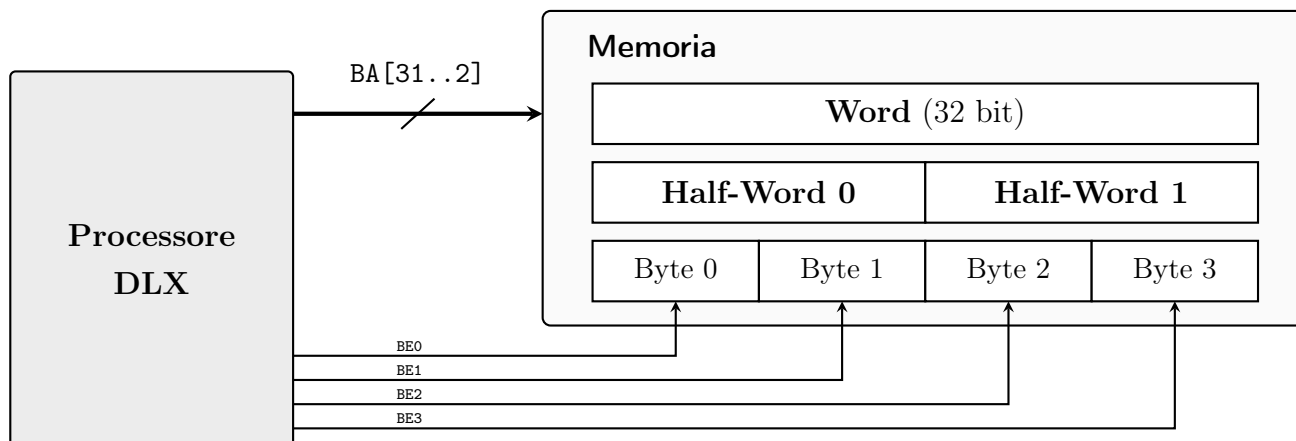


Figura 1.1: Organizzazione interna della memoria DLX e collegamento dei segnali di controllo. Il processore emette direttamente un bus indirizzi a 30 bit ($BA[31..2]$) e quattro segnali indipendenti di *Byte Enable* ($BE0$ – $BE3$), ciascuno dedicato all’abilitazione di uno specifico byte all’interno della parola selezionata.

1.1.3 Istruzioni

L’architettura *DLX* adotta un formato di istruzioni a lunghezza fissa di 32 bit [7]; durante la fase di *fetch*, ciascuna istruzione viene memorizzata all’interno dell’Instruction Register (IR). Ogni istruzione è identificata da un *opcode* di 6 bit e la struttura dei rimanenti 26 bit varia in base alla tipologia di operazione da eseguire; la codifica si suddivide in tre categorie principali:

- **Type-R (*Register*):** impiegata per operazioni aritmetico-logiche tra registri. Presenta tre campi da 5 bit ciascuno per specificare due registri sorgente e un registro destinazione, unitamente a un campo speciale di estensione da 11 bit per definire nel dettaglio l’operazione specifica.

Opcode	Rs1	Rs2	Rd	Extend
6 bit	5 bit	5 bit	5 bit	11 bit

- **Type-I (*Immediate*):** utilizzata per il trasferimento dati con la memoria, i salti condizionati e le operazioni aritmetiche con un valore immediato. Si compone di due campi da 5 bit dedicati all’indirizzamento dei registri e di un campo da 16 bit destinato all’operando immediato; quest’ultimo viene esteso in segno a 32 bit prima di essere caricato nei registri di lavoro.

Opcode	Rs1	Rd	Immediate
6 bit	5 bit	5 bit	16 bit

- **Type-J (*Jump*)**: riservata ai salti incondizionati per la modifica diretta del flusso di esecuzione espresso dal Program Counter (PC). È costituita da un unico campo da 26 bit contenente un operando immediato in complemento a due; poiché le istruzioni sono allineate a parole di 4 byte, l'offset viene esteso in segno e traslato a sinistra di 2 bit prima dell'addizione al PC, consentendo di raggiungere un intervallo di salto compreso tra:

$$-2^{25} \cdot 4 \leq \text{offset} \leq (2^{25} - 1) \cdot 4$$

Opcode	Offset
6 bit	26 bit

1.1.4 Pipelining

La struttura classica di esecuzione delle istruzioni nel *DLX* è organizzata in una *pipeline* a 5 stadi sequenziali, ognuno dei quali richiede un singolo ciclo di clock per il completamento:

1. ***Instruction Fetch*** (IF): estrazione dell'istruzione dalla memoria all'indirizzo puntato dal PC e suo caricamento nell'IR. Contestualmente, il valore del PC viene incrementato di 4 per predisporre il riferimento all'istruzione successiva.
2. ***Instruction Decode*** (ID): decodifica dell'istruzione, lettura contestuale dei due registri sorgente dal *register file* ed estensione in segno dell'eventuale campo immediato.
3. ***Execution*** (EX): esecuzione dell'operazione aritmetico-logica nella *Arithmetic Logic Unit* (ALU).
4. ***Memory Access*** (MEM): accesso in lettura o scrittura alla memoria dati, quando richiesto dall'istruzione; l'indirizzo calcolato nello stadio precedente viene inserito nel registro MAR.
5. ***Write Back*** (WB): scrittura del risultato calcolato o del dato letto all'interno del registro destinazione Rd del *register file*.

Sebbene la descrizione precedente illustri i cinque stadi come fasi sequenziali della singola istruzione, il vantaggio prestazionale della pipeline deriva dalla loro **sovrapposizione temporale** tra istruzioni consecutive: mentre un'istruzione si trova nello stadio EX, quella immediatamente successiva può contemporaneamente trovarsi nello stadio ID, e quella ancora successiva nello stadio IF. A regime, cioè una volta che la pipeline è completamente piena, il processore avvia l'esecuzione di una nuova istruzione a ogni ciclo di clock, pur mantenendo per ciascuna istruzione una latenza complessiva di 5 cicli dal fetch al write back.

In condizioni ideali, questa sovrapposizione consente di ottenere un *throughput* pari a un'istruzione completata per ciclo di clock (*Cycles Per Instruction*, $CPI = 1$), a fronte di un tempo di ciclo determinato dallo stadio più lento tra i cinque. Tale idealità viene tuttavia comunemente compromessa dalla presenza di *hazard* strutturali, di dato e di controllo, che possono imporre l'inserimento di cicli di stallo o il ricorso a tecniche di *forwarding* [8].

1.2 RISC-V

Come anticipato nelle premesse, l'architettura *RISC-V* viene analizzata in questa sede secondo una panoramica introduttiva. *RISC-V* è uno standard *open-source* di *Instruction Set Architecture* (ISA), sviluppato sul paradigma *Reduced Instruction Set Computer* (RISC) [9].

1.2.1 Origine e Mantenimento

Il progetto nasce nel 2010 presso l'Università della California, Berkeley, guidato dai professori Krste Asanović e David Patterson. Rappresenta la quinta versione dell'architettura RISC concepita dall'ateneo e si ricollega direttamente alla stirpe dei progetti storici iniziata con *RISC-I* nel 1981.

A differenza dei modelli precedenti o delle architetture commerciali proprietarie (quali x86 ed ARM), RISC-V è rilasciato sotto licenze libere e prive di royalty. La gestione e l'evoluzione dello standard sono affidate ad una fondazione globale no-profit che coordina un ecosistema industriale e accademico composto da oltre 4.500 membri.

1.2.2 Caratteristiche Architettureali Principali

L'architettura si fonda rigorosamente sul modello *load/store*, nel quale le operazioni aritmetico-logiche avvengono esclusivamente all'interno dei registri del processore, relegando l'interazione con la memoria fisica unicamente ad istruzioni dedicate di caricamento (*load*) e salvataggio (*store*).

Il set di istruzioni è strutturato secondo un approccio modulare fondato su un *Base ISA* fisso e su un insieme di estensioni opzionali [10]:

- **Spazi di Indirizzamento Graduabili:** L'ISA di base garantisce nativamente spazi di indirizzamento ed ampiezza dei registri a 32 bit (*RV32I*) e a 64 bit (*RV64I*), definendo inoltre le specifiche per una futura estensione a 128 bit (*RV128I*).

- **Istruzioni Compresse (Estensione C):** Sebbene la lunghezza standard delle istruzioni sia di 32 bit, l'estensione opzionale *Compressed* introduce codifiche a 16 bit per i comandi più frequenti.
- **Estensioni Operative (M, A, F, D):** La sintassi modulare consente di affiancare alla base intera (I) estensioni standard per la moltiplicazione/divisione hardware (M), operazioni atomiche (A), e aritmetica in virgola mobile a singola (F) e doppia precisione (D).

1.2.3 Utilizzo Industriale

Nonostante l'impronta teorica originata nel contesto accademico, l'architettura *RISC-V* è stata concepita fin dalle prime fasi di progettazione per consentire una diretta implementazione industriale. La sua natura modulare esente da costi di licenza consente di adattare la complessità dell'hardware alle specifiche del dominio di applicazione, dai microcontrollori *ultra-low-power* per dispositivi *IoT* fino ai sistemi di calcolo ad alte prestazioni per acceleratori di intelligenza artificiale.

Capitolo 2

Ammodernamento Tecnologico

Le prime attività di intervento sul simulatore hanno riguardato l'aggiornamento dello stack tecnologico sul quale è stata sviluppata l'applicazione. La struttura originaria del progetto risale al 2019 e, nel corso degli anni, le principali dipendenze software non erano state sottoposte a un analogo processo di aggiornamento. Ciò aveva progressivamente introdotto vincoli di compatibilità con gli strumenti di sviluppo più recenti e reso più complesso intervenire sul codice esistente. Prima di procedere con il redesign delle componenti del simulatore, si è quindi ritenuto necessario aggiornare le tecnologie alla base dell'applicazione.

L'intervento ha interessato principalmente tre componenti dello stack: il linguaggio TypeScript, il framework Angular e l'editor di codice CodeMirror. L'aggiornamento di queste tecnologie non è stato considerato come un'attività indipendente dal successivo redesign, poiché le versioni adottate nella nuova implementazione hanno modificato sia gli strumenti disponibili durante lo sviluppo sia le modalità di integrazione tra le diverse componenti del sistema. Le sezioni seguenti descrivono pertanto le caratteristiche delle tecnologie maggiormente rilevanti per il simulatore e gli interventi effettuati per adeguare il progetto alle versioni più recenti.

2.1 TypeScript

TypeScript è un linguaggio di programmazione open-source sviluppato da Microsoft e progettato come estensione di JavaScript. Il linguaggio è stato introdotto nel 2012 e successivamente sviluppato come alternativa tipizzata a JavaScript per applicazioni di dimensioni e complessità crescenti [11].

2.1.1 Caratteristiche Principali e Architettura

TypeScript può essere definito come un *typed superset* di JavaScript, in quanto mantiene la sintassi e le caratteristiche fondamentali del linguaggio introducendo, tra gli altri elementi, un sistema di tipizzazione statica. A differenza di JavaScript, nel quale i controlli sui tipi avvengono prevalentemente durante l'esecuzione, TypeScript consente di effettuare una parte significativa delle verifiche durante la fase di compilazione. Il codice sorgente scritto in TypeScript non viene eseguito direttamente dal browser. Il compilatore `tsc` analizza il programma e lo traduce in JavaScript secondo il target ECMAScript specificato nella configurazione del progetto. Il codice risultante può quindi essere eseguito negli ambienti JavaScript compatibili con il target selezionato.

Nel contesto del simulatore, il sistema di tipizzazione assume particolare rilevanza poiché il codice deve rappresentare strutture che corrispondono a elementi dell'architettura simulata. Registri, istruzioni, dispositivi di memoria e strutture utilizzate durante l'esecuzione possono essere descritti mediante tipi e interfacce espliciti, rendendo più vincolante il rapporto tra le diverse componenti del modello. La tipizzazione risulta inoltre utile nelle interazioni tra i moduli dell'interprete. Le operazioni di decodifica ed esecuzione delle istruzioni richiedono infatti il passaggio di strutture dati con caratteristiche definite; la possibilità di specificarne formalmente la struttura consente di individuare durante la compilazione un insieme di incompatibilità che, in assenza di tali controlli, potrebbe manifestarsi solamente durante l'esecuzione.

Un ulteriore aspetto riguarda gli strumenti messi a disposizione dall'ambiente di sviluppo. La presenza delle informazioni sui tipi permette di effettuare controlli sulle proprietà e sui metodi disponibili, facilita la navigazione tra le definizioni e rende più affidabili le operazioni di modifica del codice. Tali caratteristiche risultano particolarmente utili in un progetto caratterizzato da numerose interazioni tra componenti software differenti, come nel caso del simulatore.

2.1.2 Da Typescript 3.5.3 a Typescript 5.8.3

La versione originaria del simulatore utilizzava TypeScript 3.5.3, rilasciata nel 2019. L'aggiornamento dello stack ha portato all'adozione di TypeScript 5.8.3. Il passaggio è stato necessario anche in relazione all'aggiornamento di Angular, poiché le versioni più recenti del framework richiedono versioni compatibili del linguaggio e del relativo compilatore.

La differenza tra le due versioni non riguarda esclusivamente l'introduzione di nuove funzionalità sintattiche. Nel corso dell'evoluzione del linguaggio sono state apportate modifiche al sistema di analisi statica e alla gestione dei tipi, rendendo necessario verificare il codice esistente e adeguare le definizioni che non risultavano più compatibili con il nuovo compilatore.

Un primo effetto della migrazione ha riguardato il controllo statico dei tipi. Le versioni più recenti di TypeScript dispongono di meccanismi di analisi più articolati, in particolare nell'analisi del flusso di controllo e nella gestione dei *union types*. Nel simulatore ciò assume rilevanza nelle parti dell'interprete nelle quali una stessa struttura può assumere forme differenti in funzione del tipo di istruzione elaborata o dello stato dell'esecuzione. L'evoluzione del sistema di tipi ha inoltre ampliato le possibilità di descrivere strutture dati complesse attraverso generics, tipi condizionali e tipi ricorsivi. Questi strumenti possono essere utilizzati per esprimere in maniera più precisa le relazioni tra le strutture impiegate dall'interprete e le diverse categorie di istruzioni previste dall'ISA simulata.

Il processo di aggiornamento ha infine interessato anche il compilatore e il sistema di build. Le versioni più recenti di TypeScript hanno introdotto nel tempo diverse ottimizzazioni nella fase di compilazione e nel controllo dei tipi, mentre la configurazione del progetto ha dovuto essere adeguata alle nuove versioni del linguaggio e agli standard ECMAScript utilizzati come destinazione della transpilazione.

2.2 Angular

Angular è un framework open-source sviluppato e mantenuto da Google per la realizzazione di applicazioni web, con particolare diffusione nello sviluppo di applicazioni *single-page* (SPA) eseguite lato client [12].

2.2.1 Caratteristiche Principali e Architettura

L'architettura di Angular è basata sulla suddivisione dell'applicazione in componenti. Un componente associa una classe contenente la logica applicativa a un template HTML e, quando necessario, a uno specifico insieme di stili. La suddivisione in componenti permette di circoscrivere le responsabilità delle diverse parti dell'interfaccia e di organizzare l'applicazione secondo una struttura gerarchica. Questa organizzazione si adatta alla struttura del simulatore, la cui interfaccia comprende diverse aree con funzioni distinte. L'editor del codice, la visualizzazione dei registri, i pannelli relativi alla memoria e gli elementi dedicati alla rappresentazione dei componenti dell'architettura possono essere gestiti da componenti separati, mantenendo una relazione definita con lo stato dell'applicazione.

Un elemento centrale del framework è il collegamento tra lo stato gestito dalla logica applicativa e il template. Attraverso i meccanismi di binding e di reattività forniti da Angular, le modifiche allo stato possono essere riflesse nella rappresentazione grafica. Nel simulatore tale caratteristica è utilizzata, ad esempio, per aggiornare la visualizzazione dei registri e della memoria in seguito alle operazioni eseguite dall'interprete.

Angular integra inoltre un sistema di *Dependency Injection* (DI), attraverso il quale le dipendenze richieste da componenti e servizi possono essere fornite dal framework invece di essere istanziate direttamente dalle classi che le utilizzano. Questo meccanismo assume particolare importanza nell'architettura originale del simulatore, nella quale alcuni componenti della logica di simulazione facevano uso diretto dei servizi Angular. La relazione tra Dependency Injection e logica di dominio è stata successivamente rivalutata durante il redesign, come illustrato nel capitolo successivo.

2.2.2 Da Angular 8 ad Angular 20

La versione originaria del simulatore era basata su Angular 8, rilasciato nel 2019. L'aggiornamento alla versione 20 ha richiesto interventi distribuiti su più livelli dell'applicazione, poiché nel periodo intercorso tra le due versioni il framework ha modificato sia alcuni aspetti della propria architettura interna sia gli strumenti messi a disposizione per la realizzazione dei componenti e dei template.

Uno dei cambiamenti più rilevanti è stato il superamento del precedente *View Engine* in favore di Ivy, introdotto nel percorso evolutivo di Angular come nuovo motore di compilazione e di esecuzione. L'aggiornamento ha quindi richiesto l'adeguamento del progetto al nuovo sistema e alle modalità di compilazione adottate dalle versioni successive del framework. L'evoluzione del processo di compilazione ha inoltre introdotto nuove possibilità di ottimizzazione del codice prodotto, tra cui un utilizzo più efficace del *tree shaking*.

La migrazione ha interessato anche l'organizzazione dei componenti. Le versioni recenti di Angular consentono di definire componenti *standalone*, riducendo la necessità di organizzare le funzionalità attraverso `NgModule`. Nel progetto aggiornato questo modello è stato adottato nelle parti interessate dalla migrazione, con l'obiettivo di rendere più esplicite le dipendenze richieste dai componenti e ridurre la configurazione necessaria per la loro integrazione.

Un ulteriore cambiamento riguarda il modello di gestione dello stato e della reattività. Angular ha introdotto i *Signals* come meccanismo per rappresentare valori reattivi e per stabilire in maniera esplicita le dipendenze tra tali valori e le parti dell'interfaccia che ne utilizzano il contenuto. Nel simulatore questo meccanismo viene impiegato nelle componenti interessate dall'aggiornamento dello stato dell'architettura, consentendo di propagare le modifiche verso la relativa rappresentazione grafica.

La migrazione ha inoltre comportato l'aggiornamento della sintassi utilizzata nei template. Alle direttive strutturali tradizionalmente impiegate da Angular, come `*ngIf` e `*ngFor`, si affianca nelle versioni recenti il *built-in control flow*, basato su costrutti quali `@if`, `@for` e `@switch`.

Le parti dell'interfaccia coinvolte dalla migrazione sono state conseguentemente adeguate alla nuova sintassi.

Infine, il passaggio da Angular 8 ad Angular 20 ha richiesto la gestione delle numerose modifiche introdotte nelle versioni intermedie. L'aggiornamento ha quindi comportato la revisione delle API utilizzate dal progetto e delle relative configurazioni, oltre all'adeguamento dei componenti alle modalità di sviluppo previste dalle versioni più recenti del framework.

2.3 CodeMirror

CodeMirror è un editor di codice progettato per essere integrato all'interno di applicazioni web [13]. Nel simulatore viene utilizzato come ambiente per la scrittura e la modifica dei programmi assembly destinati alle architetture DLX e RISC-V.

2.3.1 Caratteristiche Principali e Architettura

L'utilizzo di CodeMirror permette di andare oltre le funzionalità offerte da un semplice elemento HTML `<textarea>`. L'editor mantiene infatti una rappresentazione strutturata del testo inserito e consente di associare al contenuto diverse estensioni dedicate alla gestione e alla visualizzazione del codice.

Nel simulatore una delle funzionalità principali è l'evidenziazione sintattica. Le diverse componenti del programma assembly, tra cui istruzioni, registri, label e commenti, vengono riconosciute e rappresentate secondo le regole sintattiche definite per l'architettura selezionata. Questa funzionalità è strettamente legata alla grammatica del linguaggio utilizzato e deve pertanto essere adattata alle caratteristiche degli assembly supportati dal simulatore.

Un secondo elemento è rappresentato dall'autocompletamento. Durante la scrittura del programma, l'editor può proporre i mnemonici delle istruzioni e i nomi dei registri disponibili. Poiché DLX e RISC-V presentano insiemi di istruzioni differenti, il comportamento dell'autocompletamento deve tenere conto dell'architettura attualmente selezionata dall'utente.

Queste funzionalità rendono CodeMirror una componente integrata nella logica del simulatore piuttosto che un semplice elemento destinato alla modifica del testo. La configurazione dell'editor deve infatti essere coordinata con le caratteristiche dell'assembly supportato e con le informazioni utilizzate dall'interprete.

2.3.2 Da ngx-codemirror a CodeMirror Nativo

Nella versione originaria del simulatore, l'integrazione dell'editor era realizzata attraverso **ngx-codemirror**, un wrapper Angular basato su CodeMirror 5. Tale soluzione introduceva un ulteriore livello di astrazione tra il framework e l'editor vero e proprio. Con l'aggiornamento dello stack tecnologico, questa integrazione ha evidenziato alcune limitazioni, in particolare per quanto riguarda la compatibilità con le versioni recenti di Angular, la gestione delle modifiche al DOM e la possibilità di intervenire direttamente sulla configurazione dell'editor.

La soluzione adottata è stata quindi la sostituzione di **ngx-codemirror** con un'integrazione diretta di CodeMirror 6. Questa scelta ha eliminato la dipendenza dal wrapper Angular precedentemente utilizzato e ha consentito di gestire l'editor direttamente all'interno dell'applicazione. CodeMirror 6 presenta inoltre un'architettura differente rispetto alla versione 5, caratterizzata dalla suddivisione delle funzionalità in estensioni indipendenti.

Nel nuovo modello, funzionalità quali la visualizzazione dei numeri di riga, la gestione della cronologia delle modifiche e l'evidenziazione sintattica possono essere configurate attraverso estensioni distinte. Questa organizzazione consente di comporre la configurazione dell'editor in funzione delle esigenze specifiche del simulatore, senza dover intervenire su un componente monolitico o affidarsi alle funzionalità esposte dal precedente wrapper.

La migrazione ha interessato in modo particolare l'analisi sintattica dell'assembly. Nella precedente implementazione alcune funzionalità di analisi erano basate su espressioni regolari, soluzione che risultava poco adatta alla gestione di una grammatica articolata. Con CodeMirror 6 è stato introdotto Lezer, il sistema di parsing utilizzato dall'editor, sul quale è stato sviluppato un parser specifico per le grammatiche assembly supportate dal simulatore. L'adozione di un parser dedicato consente di trattare separatamente le caratteristiche sintattiche di DLX e RISC-V e di fornire all'editor le informazioni necessarie per l'evidenziazione del codice.

La sostituzione di **ngx-codemirror** non ha quindi costituito una semplice sostituzione della versione della libreria. L'intervento ha richiesto la revisione del livello di integrazione tra l'editor e Angular, oltre alla riprogettazione delle modalità con cui il codice assembly viene analizzato e utilizzato dalle funzionalità dell'interfaccia. L'aggiornamento di CodeMirror conclude pertanto il processo di ammodernamento dello stack e costituisce la base tecnologica sulla quale sono stati successivamente sviluppati gli interventi di redesign descritti nel capitolo seguente.

Capitolo 3

Redesign e sviluppo nuove funzionalità

Successivamente all'ammodernamento dello *stack* tecnologico (descritto nel Capitolo 2), sono state intraprese le operazioni di redesign dei componenti esistenti e la progettazione di nuove funzionalità. Il presente capitolo si concentra sulle scelte architetturali e progettuali dei singoli componenti del simulatore, analizzandone l'evoluzione rispetto alla struttura originaria.

3.1 Redesign

Il redesign architetturale è stato mirato principalmente alla risoluzione del debito tecnico accumulato. Sono state tre, in particolare, le aree di analisi: ***decoupling* dei moduli** a favore di una migliore manutenibilità, **riprogettazione dei componenti** per migliorarne riusabilità/stabilità e **ottimizzazione delle prestazioni**.

Nelle sezioni che seguono, verrà affrontata l'analisi per componente o gruppo di componenti, trattando, ove necessario, le aree indicate precedentemente.

3.1.1 Memoria

Nella struttura originale del simulatore, l'architettura della memoria segue un modello a strati. Il livello di astrazione più alto è rappresentato dal servizio singleton **MemoryService** (fornito a livello *root* tramite il sistema di Dependency Injection di Angular), al quale sono delegate le operazioni di gestione ad alto livello (aggiunta, rimozione e persistenza dei dispositivi). Le operazioni di lettura e scrittura effettiva sui dati (**load/store**) sono invece implementate a un livello inferiore, all'interno della classe **Memory**, alla quale il servizio mantiene un riferimento diretto. La classe **Memory** rappresenta concretamente la memoria complessiva ed è responsabile della gestione dell'insieme di dispositivi (**Device**) che la compongono.

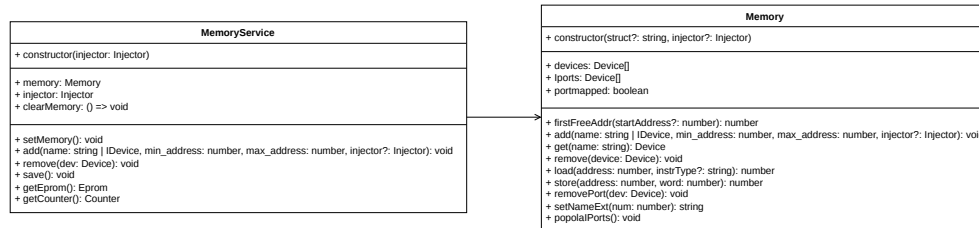


Figura 3.1: Diagramma UML originale della memoria.

Il processo di refactoring ha mantenuto tale struttura stratificata, ma è intervenuto nella gestione delle dipendenze, nella logica di dominio, nell'accesso allo stato interno e nella persistenza della memoria.

È stato in primo luogo eliminato il riferimento esplicito all'oggetto **Injector** all'interno dei costruttori e dei metodi **add**, sia in **MemoryService** sia in **Memory**, rimuovendo la dipendenza diretta dal framework nella logica di dominio. Successivamente, le firme dei metodi principali sono state riprogettate per accorpare parametri precedentemente sciolti in oggetti coerenti e tipizzati: i metodi **add**, ad esempio, passano da una firma articolata su quattro parametri a una singola istanza di **Device**, eliminando il *code smell* legato all'elevato numero di parametri. Analogamente, il metodo **load** è stato semplificato rimuovendo il parametro opzionale e inutilizzato **instrType**. È stata inoltre modificata la logica di dominio: nella nuova implementazione, il valore di ritorno del metodo rappresenta sempre una *word*.

L'accesso allo stato interno è stato limitato per migliorarne l'incapsulamento, rendendo privato il riferimento all'oggetto **Memory**. L'accesso in sola lettura alla memoria è ora possibile tramite un *getter* dedicato, che sostituisce i precedenti metodi espliciti come **getEeprom**. Le operazioni sull'architettura, quali aggiunta, rimozione e gestione degli interrupt, sono invece esposte direttamente dai metodi pubblici di **MemoryService**. I consumatori esterni possono quindi leggere lo stato corrente della memoria senza poterne riassegnare il riferimento ed eseguire le modifiche previste attraverso il servizio.

Anche la gestione della persistenza è stata riorganizzata: il generico metodo **save** del servizio, presente nella versione precedente, è stato sostituito da un insieme di metodi dedicati. In particolare, è stata integrata una serializzazione JSON, che evita che i componenti dell'interfaccia utente debbano conoscere direttamente la struttura utilizzata per rappresentare la memoria. Sono state inoltre introdotte operazioni precedentemente assenti o inserite in altri metodi, quali **init**, utilizzato per inizializzare la memoria a uno stato predefinito oppure per ripristinare lo stato salvato in **localStorage**, **interrupt**, che centralizza la gestione delle interruzioni hardware generate dai dispositivi, e **findDevice**, che permette di individuare in **Memory** un dispositivo a partire dal relativo indirizzo.

Infine, il processo di refactoring ha comportato anche l'eliminazione di elementi non più necessari, tra cui i metodi `clearMemory`, `setMemory` e `getCounter` (in `MemoryService`) e la proprietà `portmapped` con i metodi `removePort` e `popolaIPorts` (in `Memory`). La loro rimozione riduce il numero di metodi e proprietà esposti verso l'esterno.

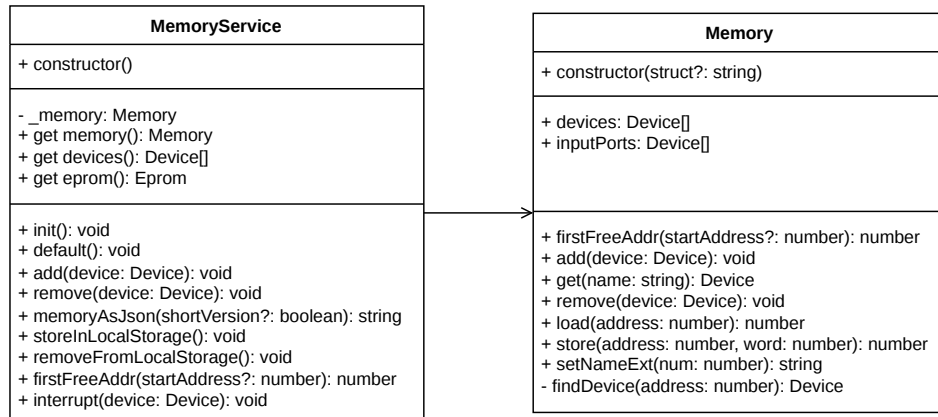


Figura 3.2: Diagramma UML riprogettato della memoria.

Molte delle scelte progettuali qui descritte non sono isolate, ma derivano direttamente dalla parallela riprogettazione dei componenti `Device`, approfondita nella sottosezione 3.1.2. La semplificazione delle firme dei metodi di `Memory` e `MemoryService`, in particolare l'accorpamento dei parametri di `add` in una singola istanza tipizzata, è infatti resa possibile dall'introduzione di una gerarchia di dispositivi più espressiva e autoconsistente, capace di incapsulare al proprio interno le informazioni prima passate come argomenti separati. Allo stesso modo, la serializzazione JSON centralizzata è resa possibile dall'implementazione, per ogni `Device`, di un metodo di serializzazione automatica.

Nella forma originale, la gerarchia **Device** contiene diverse dipendenze verso componenti esterni alla logica dei dispositivi.

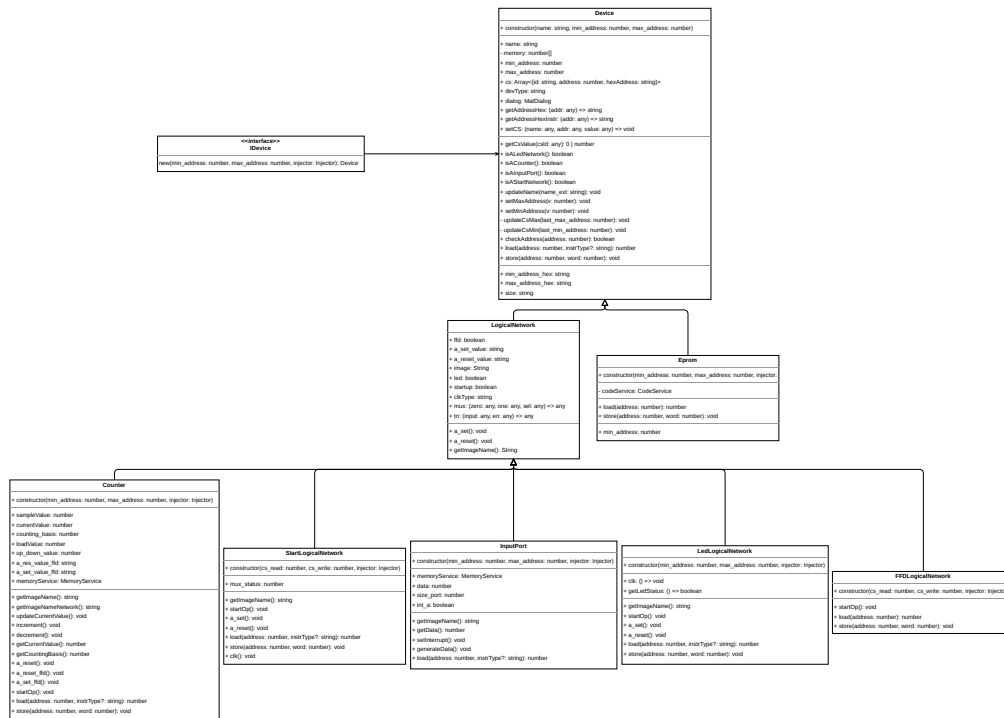


Figura 3.3: Diagramma UML originale dei componenti di memoria.

Nel caso di `MemoryService` e `Memory`, il redesign ha riguardato principalmente l'interfaccia pubblica. Per la gerarchia `Device`, il problema principale riguardava invece le dipendenze tra le classi.

Nella versione originale, classi come `Counter` e `InputPort` mantenevano un riferimento diretto a `MemoryService`, introducendo un accoppiamento tra i singoli dispositivi e il servizio che li gestisce. Questa struttura poteva inoltre generare una dipendenza circolare del tipo `Device` $\rightarrow$ `MemoryService` $\rightarrow$ `Memory` $\rightarrow$ `Device`. Un problema analogo era presente nella classe base `Device`, che deteneva un riferimento diretto al servizio di *Angular Material* utilizzato per la gestione delle finestre di dialogo. In questo modo, il modello dei dispositivi hardware dipendeva direttamente da un componente dell'interfaccia grafica. Entrambi i riferimenti sono stati rimossi nella versione riprogettata. È stata inoltre rimossa l'istanza di `Injector`, precedentemente richiesta dai costruttori della gerarchia, sostituendola con parametri espliciti e tipizzati. Queste modifiche riducono le dipendenze tra il modello dei dispositivi e i componenti esterni, permettendo di utilizzare le classi della gerarchia senza dover coinvolgere direttamente i servizi Angular.

È stata inoltre modificata la rappresentazione dello stato interno dei dispositivi di memoria. Il campo `memory`, precedentemente un semplice array generico, è stato ri-tipizzato come `Uint32Array`, una *typed array* nativa che riduce l'occupazione di memoria e migliora le prestazioni di accesso. A questa modifica si aggiunge l'introduzione dei campi `_initialized` e `_seed`, insieme ai metodi `isInitialized`, `markInitialized` e `_fastRandom`, utilizzati per tenere traccia delle celle di memoria che non sono ancora state scritte. Le motivazioni e gli effetti di questa modifica sono analizzati nella sezione 3.3.

Anche il sistema di serializzazione della gerarchia, come introdotto nella sottosezione 3.1.1, è stato modificato. Ogni classe implementa ora, in modo uniforme, la coppia di metodi `toJSON/fromJSON`, insieme al metodo protetto `hydrate` per il ripristino dello stato interno. La responsabilità della creazione di un dispositivo, precedentemente affidata alla combinazione di parametri grezzi e `Injector` tramite la factory `IDevice`, è stata spostata nella deserializzazione di una rappresentazione JSON tramite `DeviceFactory`. Questa soluzione mantiene la gestione della serializzazione allineata al sistema di persistenza descritto nella sezione precedente.

Lo stesso approccio è stato utilizzato per il campo `cs` di `Device`, che nella versione originale era rappresentato da un semplice array di oggetti letterali anonimi. È stata quindi introdotta la classe dedicata `ChipSelect`, dotata di costruttore, *factory* statica (`ChipSelect.of`) e metodi propri di serializzazione.

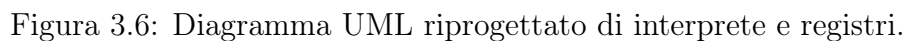
Sono state infine apportate alcune modifiche di carattere manutentivo. La classe base `Device` esponeva in precedenza una serie di metodi nella forma `isA...()`, utilizzati per ottenere manualmente informazioni sul tipo dell'oggetto a runtime (*RTTI*). Tali metodi sono stati rimossi dalla nuova gerarchia. Le convenzioni di denominazione sono state inoltre uniformate su tutto il codice, eliminando l'inconsistenza tra *snake_case* e *camelCase* presente nella versione originale. Alcuni metodi condivisi da più sottoclassi, tra cui `extractByte`, `positionValue` e `startOperation`, sono stati infine spostati nella classe base `LogicalNetwork`, centralizzando la logica comune ed evitando la duplicazione nelle sottoclassi.

È stata inoltre introdotta la classe `Ram`, che rappresenta esplicitamente il dispositivo di memoria RAM come sottoclasse di `Device`, analogamente a quanto già avveniva per `Eprom`. Quest'ultima è stata a sua volta modificata rimuovendo il riferimento privato a `CodeService`, non più presente nella versione riprogettata.

la mappa `specialRegisters` per quelli che richiedono una gestione specifica. La mappa contiene oggetti `SpecialRegisterDescriptor`, attraverso i quali ogni architettura può definire i propri registri speciali e i relativi metadati. Questa struttura consente di utilizzare la stessa logica di gestione e di tracciamento nelle diverse implementazioni dei registri. La stessa struttura è stata utilizzata per il flag `interruptEnabled`, che nella versione precedente era definito come campo separato della classe `Interpreter`. Nella nuova implementazione il valore è stato inserito nella mappa `specialRegisters`, trattandolo quindi secondo lo stesso meccanismo utilizzato per gli altri registri speciali. È stato inoltre rimosso il metodo `handleOverflow`, che nella versione precedente si limitava a rilanciare l'eccezione ricevuta senza effettuare ulteriori elaborazioni.

Il redesign ha interessato anche i tipi utilizzati durante l'esecuzione delle istruzioni. Strutture precedentemente definite in forma anonima o direttamente all'interno delle dichiarazioni sono state sostituite da tipi espliciti. In particolare, `DLXInstructionContext` rappresenta il contesto passato ai gestori delle istruzioni, `InstructionHandler` ne definisce la firma e `DLXStructuredInstruction` specifica il tipo restituito dal metodo `processLine()`. Il campo `process_instruction`, precedentemente caratterizzato da una tipizzazione meno restrittiva, è stato definito come `Record<DLXInstructionType, InstructionHandler>`. È stato inoltre modificato il tipo `InstructionType`. Le sigle utilizzate nella versione originale sono state sostituite da identificatori descrittivi nel nuovo tipo `DLXInstructionType`. La modifica riguarda esclusivamente la rappresentazione del tipo nel codice e non il comportamento delle istruzioni durante l'esecuzione. L'elenco dei mnemonici supportati (`ADD`, `ADDI`, `SW`, `BEQZ`, ...), già definito nella versione originale tramite il tipo `Instruction`, è rimasto invariato.

È stata modificata anche l'interfaccia pubblica dell'interprete. Il metodo `run()` è stato rinominato in `execute()` nelle classi `DLXInterpreter` e `RV32Interpreter` e nell'interfaccia `Interpreter`. Il tipo di ritorno è stato poi cambiato da `void` a `number`, a rappresentare l'indirizzo in memoria della prossima istruzione. Sono stati inoltre introdotti i metodi `reset` e `resetArchitectureState`, utilizzati per rendere esplicita e centralizzare la procedura di ripristino dello stato dei registri.



22

3.2 Sviluppo delle Nuove Funzionalità

Parallelamente all'attività di refactoring, sono state introdotte diverse nuove funzionalità, alcune rivolte all'utilizzo del simulatore durante le sessioni di simulazione e altre alla gestione e alla manutenzione del sistema. Queste funzionalità sono state sviluppate nel contesto del redesign dell'interfaccia grafica, descritto nel Capitolo 4. Nelle sottosezioni seguenti vengono descritte le principali funzionalità introdotte.

3.2.1 Comandi di Esecuzione e Scorciatoie da Tastiera

Per velocizzare il controllo dell'esecuzione durante le sessioni di simulazione, è stata introdotta la possibilità di invocare i principali comandi tramite scorciatoie da tastiera. Le operazioni di avanzamento passo-passo (*step*), esecuzione continua (*run*) e arresto (*stop*) sono state associate rispettivamente ai tasti funzione F8, F9 e F10.

Le scorciatoie permettono di eseguire queste operazioni senza utilizzare i relativi controlli dell'interfaccia grafica. In particolare, risultano utili durante le attività di debug in cui è necessario eseguire ripetutamente singole istruzioni e controllare lo stato dei registri tra un'esecuzione e la successiva.

3.2.2 Starting Script

Per facilitare l'utilizzo iniziale del simulatore, è stato introdotto un meccanismo di *starting script*. La funzionalità consente di caricare direttamente nell'ambiente di esecuzione alcuni programmi di esempio, evitando di dover scrivere o importare manualmente uno script prima di iniziare una sessione.

La piattaforma mette a disposizione una selezione di programmi predefiniti, tra cui:

- **Calcolo della successione di Fibonacci:** uno script che mostra l'utilizzo dei cicli, delle variabili temporanee e dell'iterazione su valori sequenziali.
- **Somma degli elementi di un array:** un algoritmo che utilizza l'accesso alla memoria tramite indirizzamento indiretto e lo scorrimento degli elementi di un vettore.

Questi script possono essere utilizzati come esempi durante l'esecuzione del simulatore e come base per verificare il comportamento delle componenti di decodifica ed esecuzione dell'interprete.

3.2.3 Sistema di Segnalazione Anomalie

È stato inoltre integrato un sistema per la segnalazione di anomalie riscontrate durante l'utilizzo del simulatore. La funzionalità consente di generare e scaricare un'istantanea (*dump*) contenente

lo stato della memoria e il codice sorgente attualmente caricato. L'utente può inoltre inviare una segnalazione tramite e-mail, associando al *dump* una descrizione del comportamento osservato.

L'implementazione di questa funzionalità utilizza il sistema di serializzazione introdotto durante la riprogettazione dei componenti **Memory** e **Device**, descritta nelle sottosezione 3.1.1 e sottosezione 3.1.2. La possibilità di serializzare i singoli componenti in formato JSON permette infatti di ricostruire lo stato della memoria e dei dispositivi al momento della segnalazione.

3.3 Valutazione delle Prestazioni e Ottimizzazione della Memoria

Come anticipato nella sottosezione 3.1.2, gli interventi sui componenti **Device**, in particolare quelli dotati di memoria interna, hanno avuto effetti sui tempi di caricamento dell'applicazione e sulla dimensione del pacchetto distribuito.

Prendendo come riferimento la versione originale del simulatore, l'indice di *Largest Contentful Paint* (LCP) si attestava su un valore pari a 1,89 s. Durante le prime fasi di sviluppo e sperimentazione, è stata valutata l'ipotesi di inizializzare in maniera totalmente aleatoria l'intero spazio di memoria al momento dello startup. Questa soluzione ha però portato il tempo di LCP a superare talvolta i 17 s in assenza di meccanismi di *caching*. Sulla base di questi risultati, la randomizzazione è stata spostata al momento del primo accesso alla singola parola di memoria. La strategia *lazy* distribuisce quindi il costo dell'inizializzazione durante l'utilizzo della memoria, evitando di elaborare l'intero spazio di memoria durante l'avvio dell'applicazione. Per tenere traccia delle celle già inizializzate sono state introdotte le funzionalità **isInitialized** e **markInitialized**. Con questa implementazione, il valore di LCP misurato è sceso a 0,92 s.

Oltre ai tempi di caricamento, sono stati analizzati anche i tempi di compilazione e la dimensione dell'output generato. Nel sistema originale, il processo di compilazione richiedeva circa 34,91 s e produceva una struttura di *chunk* caratterizzata dalla doppia emissione di pacchetti compatibili con i target **es2015** ed **es5**. Il solo modulo *vendor* occupava uno spazio grezzo compreso tra gli 8,22 MB e i 9,52 MB, a cui si aggiungevano circa 1,55 MB per il codice applicativo principale e oltre 2 MB ripartiti tra stili e *polyfills*. A seguito della riprogettazione e dell'adozione delle strategie di *bundling* disponibili nella nuova versione di Angular, l'output di build è stato unificato. La dimensione grezza totale dei file iniziali è scesa a 3,06 MB, suddivisi principalmente tra 2,73 MB dedicati al pacchetto principale (**main**), 293,38 kB per gli stili e 34,61 kB per i *polyfills*. Considerando la compressione applicata durante il trasferimento via rete, la dimensione stimata di download è pari a 702,29 kB. Il tempo necessario per completare

il processo di generazione dell'applicazione è inoltre sceso a 11,78 s. La Tabella 3.1 riassume il confronto tra le due versioni.

Metrica	Versione originale	Versione riprogettata
<i>Largest Contentful Paint</i> (LCP)	1,89 s	0,92 s
Tempo di build	34,91 s	11,78 s
Dimensione totale iniziale (grezza)	~ 11,77–13,07 MB*	3,06 MB
di cui pacchetto <i>vendor</i>	8,22–9,52 MB	—**
di cui pacchetto principale	1,55 MB	2,73 MB
di cui stili	~ 2 MB (stili e polyfill)	293,38 kB
di cui polyfill		34,61 kB
Dimensione stimata di download (compressa)	—***	702,29 kB

* Somma delle dimensioni di *vendor*, codice applicativo e stili/polyfill.

** Nella nuova configurazione di *bundling*, il codice di terze parti non viene più emesso come *chunk* separato, ma unificato nel pacchetto principale.

*** Dato non disponibile per la versione originale.

Tabella 3.1: Confronto delle principali metriche di build e caricamento tra la versione originale e quella riprogettata del simulatore.

Capitolo 4

Interfaccia Grafica

La riprogettazione dell'interfaccia grafica è stata avviata a partire dai limiti riscontrati durante l'utilizzo della versione originale del simulatore. In particolare, la disposizione dei controlli di esecuzione, l'organizzazione della sezione dedicata ai registri e la distinzione visiva tra i diversi elementi della memoria rendevano meno immediata la lettura dello stato della simulazione. Gli interventi descritti in questo capitolo hanno riguardato principalmente la palette cromatica, il posizionamento dei comandi, la rappresentazione dei registri e la gestione delle finestre di dialogo.

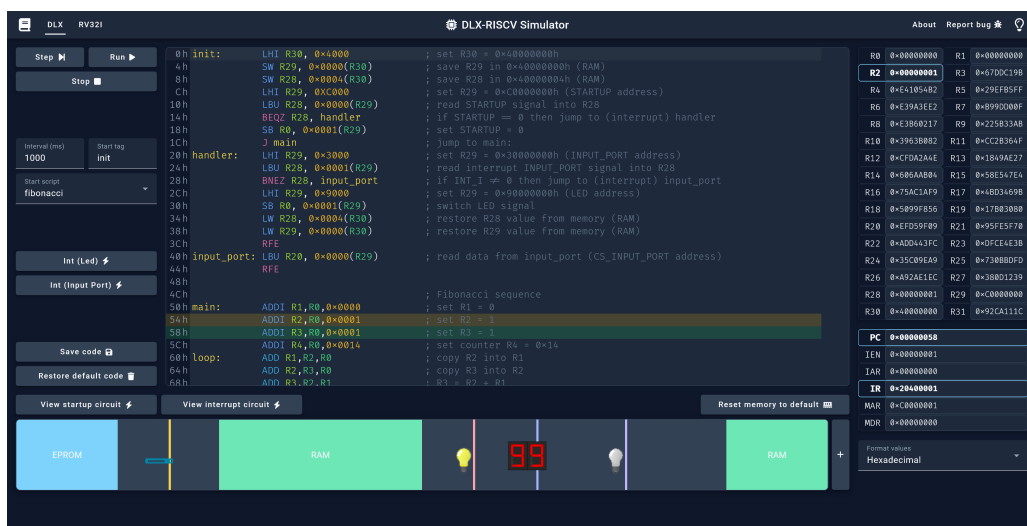


Figura 4.1: Interfaccia del simulatore dopo il processo di riprogettazione.

4.1 Palette e Identità Visiva

La palette originaria del simulatore, nella sua variante scura, era basata su quattro tonalità cromatiche principali. Il colore #303030 veniva utilizzato come sfondo generale dell'interfaccia, con variazioni più chiare per i pulsanti e più scure per l'editor. La tonalità #2196f3 era impiegata nell'header e in alcuni elementi associati alla memoria, mentre il verde #b2ff59 e il rosso #ff0000 erano rispettivamente associati alle azioni primarie e alle azioni critiche.

Nella nuova versione la palette è stata riorganizzata mediante l'impiego di tonalità meno saturate e di una distinzione più uniforme tra le diverse superfici dell'interfaccia. Il colore #1e293b è stato adottato per l'header, l'editor di codice e la sezione dei registri, mentre la tonalità neutra #3b485c è stata utilizzata per la superficie dei pulsanti.

Per la rappresentazione dei diversi blocchi di memoria sono state introdotte tonalità pastello, tra cui l'azzurro per la EPROM e il verde acqua per la RAM. Inoltre, la riga corrispondente all'istruzione in esecuzione nell'editor viene evidenziata mediante una barra orizzontale a contrasto, estesa per l'intera larghezza del pannello.

Le varianti relative al tema chiaro sono state definite a partire dalla palette del tema scuro, mantenendo una corrispondenza tra i colori delle superfici e quelli degli elementi funzionali presenti nelle due modalità.

4.2 Pulsanti e Controlli

L'interfaccia del simulatore comprende diversi controlli destinati alla gestione dell'esecuzione, alla configurazione della simulazione e all'interazione con i componenti hardware. Il restyling ha previsto una revisione uniforme dell'aspetto e della disposizione di tali elementi. I pulsanti non utilizzano più i colori associati alle singole azioni nella versione precedente, ma adottano una finitura neutra accompagnata da un'iconografia bianca comune. È stato inoltre rimosso il pulsante dedicato alla visualizzazione dei diagrammi relativi ai cicli di bus.

Per quanto riguarda la disposizione dei controlli, i comandi relativi all'esecuzione sono stati collocati nella barra laterale sinistra e suddivisi in moduli distinti. Nella parte superiore sono presenti le azioni di esecuzione passo-passo (*Step*), esecuzione continua (*Run*) e arresto (*Stop*). A queste seguono i controlli relativi alla temporizzazione della simulazione e il menu a tendina per la selezione degli *starting script*. Nella parte inferiore della barra sono state collocate le azioni per il salvataggio del codice e per l'ispezione dei circuiti.

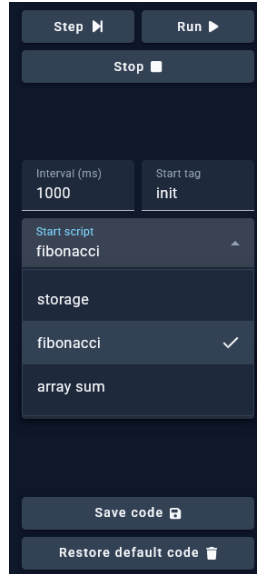


Figura 4.2: Controlli per la gestione dell'esecuzione.

Questa organizzazione consente di separare i controlli operativi dalle altre sezioni dell'interfaccia e di lasciare all'editor una maggiore disponibilità di spazio verticale per la visualizzazione del codice sorgente.

I controlli relativi alla gestione della memoria sono stati invece posizionati in prossimità dei corrispondenti componenti hardware. In questa sezione sono stati aggiunti un pulsante per il ripristino dello stato iniziale della memoria e un comando per l'ispezione del circuito di startup.

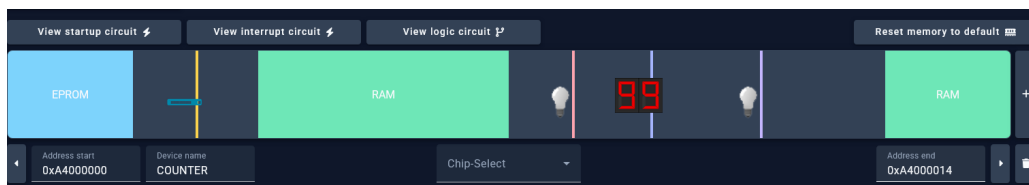


Figura 4.3: Controlli per la gestione della memoria.

È stata inoltre modificata la modalità di ispezione della memoria. Le precedenti azioni *show* e *show detail* sono state sostituite da un'unica operazione denominata *inspect*. A partire dall'indirizzo specificato, questa funzione consente di esaminare quattro parole di memoria consecutive e di visualizzare le relative rappresentazioni in *byte*, *half-word* e *word*. Per ciascuna rappresentazione sono disponibili i formati esadecimale, decimale e binario.

Memory values			
0x40000000	0x40000001	0x40000002	0x40000003
0x4E	0x4A	0x5F	0x76
0x40000004	0x40000005	0x40000006	0x40000007
0xB1	0x37	0x29	0x24
0x40000008	0x40000009	0x4000000A	0x4000000B
0xD6	0x63	0xE6	0x8D
0x4000000C	0x4000000D	0x4000000E	0x4000000F
0xFC	0xF0	0x69	0x54
✓	Hex	Dec	Bin
✓	Byte	Halfword	Word
Cancel	Done		

Figura 4.4: Ispezione della memoria all'indirizzo specificato con campi modificabili.

Infine, all'interno della barra *header* è stata aggiunta una procedura per la segnalazione delle anomalie.

4.3 Registri

Nella nuova versione del simulatore, la sezione dedicata ai registri mantiene la propria collocazione sul lato destro dell'editor, ma presenta una diversa organizzazione degli elementi. Nella versione originale i *General Purpose Registers* erano disposti in un'unica colonna stretta, estesa prevalentemente in senso verticale. La nuova interfaccia organizza invece i GPR su due colonne affiancate, consentendo di visualizzare un numero maggiore di registri nella stessa porzione verticale dell'interfaccia, aumentando solo leggermente lo spazio orizzontale occupato.

I registri speciali sono stati separati dai GPR e raccolti in un pannello dedicato, posizionato immediatamente al di sotto di questi ultimi. Il passaggio del cursore del mouse sopra un elemento, poi, determina la visualizzazione di un riquadro informativo contenente il nome completo del registro.

È stato inoltre modificato il feedback visivo associato all'esecuzione delle istruzioni. Nella versione precedente, il registro modificato durante l'ultimo passo di simulazione veniva identificato principalmente mediante il testo in grassetto. Nella nuova interfaccia, lo stesso registro viene evidenziato attraverso un riquadro ad alto contrasto con contorno azzurro. Questa soluzione rende più immediata l'individuazione del registro il cui valore è cambiato durante l'ultima operazione eseguita.

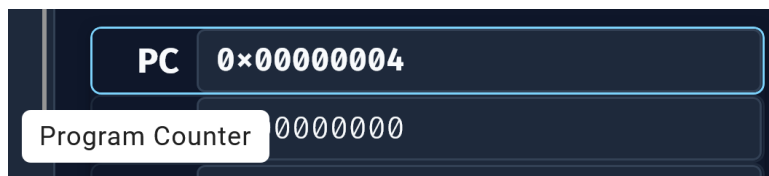


Figura 4.5: Registro speciale ed evidenziamento modifica registro.

4.4 Dialog

Un ulteriore intervento ha riguardato le finestre di dialogo utilizzate per interagire con il modello di dominio. In particolare, tali finestre vengono impiegate durante le operazioni di ispezione e configurazione dei circuiti logici associati agli elementi di memoria.

Rispetto alla versione originale, le informazioni e i parametri operativi disponibili nei dialog sono stati mantenuti, ma riorganizzati all'interno di una struttura più uniforme. L'intervento ha riguardato principalmente la disposizione dei contenuti, l'organizzazione dei campi e la definizione degli elementi di interazione. Anche la tipologia e lo stile dei campi di input sono stati uniformati, in modo da rendere omogenea la presentazione delle diverse finestre di dialogo.

Oltre alla revisione grafica, è stato modificato il comportamento con cui le variazioni inserite dall'utente vengono applicate allo stato del simulatore. In ciascun *dialog* sono state introdotte esplicitamente le azioni *Conferma* e *Cancella*.

Nella versione originale, le modifiche inserite nei campi potevano riflettersi immediatamente sullo stato dell'applicazione. La nuova implementazione adotta invece un meccanismo di modifica differita: durante la compilazione dei campi, gli elementi di memoria e i dispositivi hardware sottostanti non vengono modificati direttamente. La configurazione corrente rimane invariata fino a quando l'utente non seleziona esplicitamente l'azione di conferma.

L'azione *Cancella* consente invece di interrompere l'operazione senza applicare le modifiche effettuate all'interno della finestra. In questo modo, l'aggiornamento dello stato del simulatore avviene soltanto a seguito di una scelta esplicita dell'utente, riducendo il rischio di modifiche accidentali alla configurazione della memoria e dei dispositivi hardware.

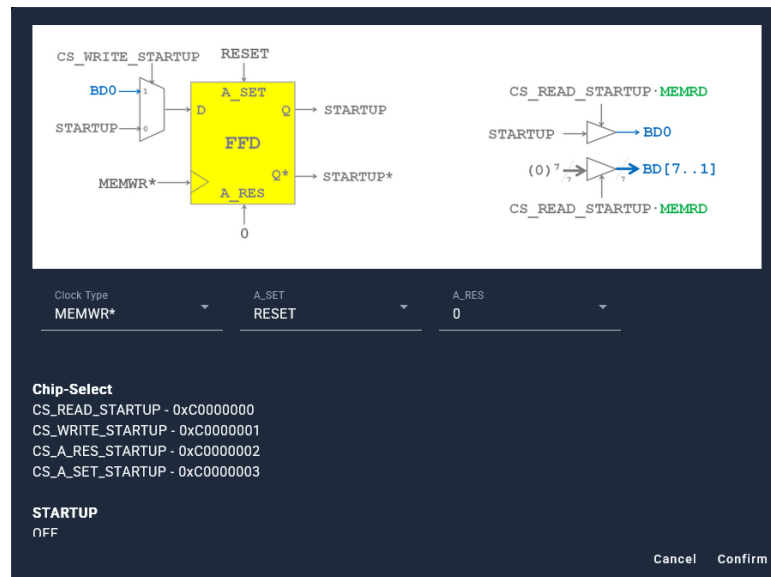


Figura 4.6: Finestra per la modifica di una rete di startup.

Capitolo 5

Sviluppi Futuri

Il lavoro di refactoring e le ottimizzazioni descritte nei capitoli precedenti hanno modificato la struttura interna del simulatore e ne hanno migliorato le prestazioni. Restano tuttavia alcuni interventi che possono essere sviluppati in futuro, sia per estendere le funzionalità didattiche del simulatore sia per aumentare la copertura dei test e la capacità di simulazione.

Un primo sviluppo riguarda l'introduzione di una suite di test automatici. Il refactoring ha separato maggiormente la logica di esecuzione dai componenti dell'interfaccia e ha reso più indipendenti i singoli moduli. Su questa struttura possono essere implementati test unitari e di integrazione per verificare il comportamento dell'interprete, le conversioni relative alla memoria e le operazioni aritmetico-logiche. I test permetterebbero inoltre di individuare eventuali regressioni introdotte da modifiche successive.

Un ulteriore intervento può riguardare il sistema di gestione della memoria. Le modifiche introdotte per l'inizializzazione dinamica delle celle e per la tracciabilità degli indirizzi possono essere estese con meccanismi di allocazione e segmentazione più dettagliati. Un possibile sviluppo consiste nella simulazione parametrizzabile di strutture di *cache* a più livelli, con differenti politiche di rimpiazzo e di scrittura.

Anche il modulo dei diagrammi può essere ulteriormente riprogettato. Il sistema attuale, in parte ereditato dalle versioni precedenti, può essere semplificato e suddiviso in componenti maggiormente indipendenti. Questo permetterebbe di migliorare la visualizzazione dei flussi dei dati e dei segnali di controllo durante l'esecuzione delle istruzioni.

Un ulteriore sviluppo riguarda la visualizzazione della *pipeline*. La separazione introdotta nell'interprete permette di considerare le diverse fasi del ciclo di istruzione (*Fetch*, *Decode*, *Execute*, *Memory*, *Write Back*) e di rappresentarne l'avanzamento durante la simulazione. Il sistema potrebbe inoltre mostrare la gestione dei *data hazard* e dei *control hazard*. In questo

caso, sarebbe necessario estendere il controllo dell'esecuzione per consentire l'avanzamento non solo per singola istruzione (*step*), ma anche per singolo ciclo di *clock*.

Conclusioni

Il lavoro di tesi ha riguardato sia l'introduzione di nuove funzionalità nel simulatore sia la revisione dell'infrastruttura esistente. In particolare, l'attività di refactoring ha avuto l'obiettivo di intervenire sull'organizzazione architetturale del sistema, migliorandone la manutenibilità e predisponendo la struttura del software a successive estensioni.

Dal punto di vista logico e applicativo, la separazione delle responsabilità tra il modello di dominio e i dettagli dell'interfaccia utente ha permesso di ridurre il livello di accoppiamento tra i diversi componenti del simulatore. A questo intervento si sono aggiunti la rimozione delle dipendenze globali, la definizione di un meccanismo di serializzazione in formato JSON e la revisione della gestione della memoria mediante strutture *typed array* e strategie di inizializzazione *lazy*. Nel complesso, queste modifiche hanno contribuito a migliorare la gestione delle risorse e i tempi di caricamento, oltre a rendere più agevoli le operazioni di manutenzione e di estensione del codice.

Parallelamente, la riprogettazione dell'interfaccia grafica ha interessato la palette cromatica, la disposizione dei controlli, la rappresentazione dei registri e l'organizzazione delle finestre di dialogo. La riorganizzazione dei comandi di esecuzione, la disposizione dei registri su più colonne e l'introduzione di un meccanismo di modifica differita nei dialog hanno consentito di rendere più chiara la presentazione delle informazioni e di limitare l'applicazione involontaria delle modifiche alla configurazione del simulatore.

Nel complesso, gli interventi realizzati hanno portato a una revisione dell'infrastruttura software e dell'interfaccia del simulatore, mantenendone la funzione di strumento didattico per lo studio dell'architettura DLX. Il risultato costituisce una base sulla quale sviluppare ulteriori funzionalità, introdurre nuove architetture didattiche e proseguire l'evoluzione del simulatore.

Bibliografia

- [1] G. Piazzzi, “Simulatore del processore DLX per scopi didattici,” tesi di laurea, Alma Mater Studiorum - Università di Bologna, 2022. A.A. 2021/2022 – [\[PDF online\]](#).
- [2] U. Laghi, “Estensione del simulatore del processore DLX,” tesi di laurea, Alma Mater Studiorum - Università di Bologna, 2022. A.A. 2021/2022 – [\[PDF online\]](#).
- [3] F. Comastri, “Estensione di un simulatore del processore DLX,” tesi di laurea, Alma Mater Studiorum - Università di Bologna, 2021. A.A. 2020/2021 – [\[PDF online\]](#).
- [4] F. Pomponii, “Sviluppo di un simulatore DLX per scopi didattici,” tesi di laurea, Alma Mater Studiorum - Università di Bologna, 2020. A.A. 2019/2020 – [\[PDF online\]](#).
- [5] F. Maccagnani, “Progetto di un simulatore di DLX per scopi didattici,” tesi di laurea, Alma Mater Studiorum - Università di Bologna, 2019. A.A. 2018/2019 – [\[PDF online\]](#).
- [6] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*. Morgan Kaufmann, 1st ed., 1990.
- [7] E. Miller, “The DLX Processor.” [\[Link online\]](#), 1996. University of Maryland, Baltimore County (UMBC) – CMSC 411, Computer Architecture Course Material.
- [8] CARV Lab, ICS-FORTH, “Asynchronous Open-Source DLX Processor (ASPIDA).” [\[Link online\]](#), 2005.
- [9] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design RISC-V Edition: The Hardware/Software Interface*. Morgan Kaufmann, 1st ed., 2017.
- [10] A. Waterman and K. Asanović, *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*. RISC-V International, 2019. [\[Specifiche online\]](#).
- [11] G. Bierman, M. Abadi, and M. Torgersen, “Understanding TypeScript,” in *Proceedings of the 28th European Conference on Object-Oriented Programming (ECOOP 2014)*, vol. 8586 of *Lecture Notes in Computer Science*, (Berlin, Heidelberg), pp. 257–281, Springer, 2014.

- [12] A. Bampakos and P. Deeleman, *Learning Angular: A no-nonsense guide to building web applications with Angular*. Packt Publishing, 4th ed., 2023.
- [13] M. Haverbeke, “CodeMirror System Guide.” [\[Link online\]](#), 2022. CodeMirror Architecture and System Design Documentation – consultato il 15 marzo 2026.